

Week-1

A) Write a program to implement perceptron with Single Input

```
x = list(map(int, input("Enter x:").split()))
y = list(map(int, input("Enter y:").split()))
w, b = map(int, input("Enter Weight and Bias:").split())
n = float(input("Enter Learning rate:"))
t = int(input("Enter Threshold value:"))

def step(z):
    return 1 if z>=t else 0

actual = [0 for i in range(len(x))]
print("Epoch: 1", "Weight:", w, "Bias:", b)
epoch = 2

while (y != actual):
    for i in range(len(x)):
        z = w*x[i] + b
        y1 = step(z)
        error = y[i] - y1
        if(error != 0):
            w = w + n * error * x[i]
            b = b + n * error
            print("    Sample:", i+1, "ERROR ", w, b)
        else:
            print("    Sample:", i+1, "No error")
        actual[i] = y1
    print("Epoch:", epoch, "Weight:", w, "Bias:", b)
    epoch += 1

print("Final:\n", "Weight:", w, "Bias:", b)
```

Output:

```
Enter x: 0 1 2 3
Enter y: 0 0 1 1
Enter Weight and Bias: 1 0
Enter Learning rate: 0.5
Enter Threshold value: 1
Epoch: 1 Weight: 1 Bias: 0
    Sample: 1 No error
    Sample: 2 ERROR 0.5 -0.5
    Sample: 3 ERROR 1.5 0.0
    Sample: 4 No error
Epoch: 2 Weight: 1.5 Bias: 0.0
    Sample: 1 No error
    Sample: 2 ERROR 1.0 -0.5
    Sample: 3 No error
    Sample: 4 No error
```

Epoch: 3 Weight: 1.0 Bias: -0.5

Sample: 1 No error

Sample: 2 No error

Sample: 3 No error

Sample: 4 No error

Epoch: 4 Weight: 1.0 Bias: -0.5

Final:

Weight: 1.0 Bias: -0.5

B) Write a program to implement perceptron with Two Inputs

```
x1 = list(map(int, input("Enter x1: ").split()))
x2 = list(map(int, input("Enter x2: ").split()))
y = list(map(int, input("Enter y: ").split()))
w1, w2 = map(int, input("Enter Weights: ").split())
b = int(input("Enter Bias: "))
n = float(input("Enter Learning rate: "))
t = int(input("Enter Threshold: "))

def step(z):
    return 1 if z>=t else 0

actual = [0 for i in range(len(x))]
print("Epoch: 1", "Weight:", w1, w2, "Bias:", b)
epoch = 2

while (y != actual):
    for i in range(len(x)):
        z = w1*x1[i] + w2*x2[i] + b
        y1 = step(z)
        error = y[i] - y1
        if(error != 0):
            w1 = w1 + n * error * x1[i]
            w2 = w2 + n * error * x2[i]
            b = b + n * error
            print("    Sample:",i+1,"ERROR ", w1, w2, b)
        else:
            print("    Sample:",i+1,"No error")
        actual[i] = y1
    print("Epoch:", epoch, "Weight:", w1, w2, "Bias:", b)
    epoch += 1

print("Final:\n", "Weight:", w1, w2, "Bias:", b)
```

Output:

Enter x1: 0 0 1 1

Enter x2: 0 1 0 1

Enter y: 1 0 0 0

Enter Weights: 0 -1

Enter Bias: 1

Enter Learning rate: 0.5

Enter Threshold: 1

G. NARAYANAMMA INSTITUTE OF TECHNOLOGY & SCIENCE

Epoch: 1 Weight: 0 -1 Bias: 1
Sample: 1 No error
Sample: 2 No error
Sample: 3 ERROR -0.5 -1.0 0.5
Sample: 4 No error
Epoch: 2 Weight: -0.5 -1.0 Bias: 0.5
Sample: 1 ERROR -0.5 -1.0 1.0
Sample: 2 No error
Sample: 3 No error
Sample: 4 No error
Epoch: 3 Weight: -0.5 -1.0 Bias: 1.0
Sample: 1 No error
Sample: 2 No error
Sample: 3 No error
Sample: 4 No error
Epoch: 4 Weight: -0.5 -1.0 Bias: 1.0
Final:
Weight: -0.5 -1.0 Bias: 1.0

Week-2

A) Write a program to implement AND, OR gate using Perceptron.

```
x1=[0,0,1,1]
x2=[0,1,0,1]
y=[0,0,0,1]
w1=1.2
w2=0.6
n=0.5
t=1
def step(z):
    return 1 if z>=t else 0
actual=[0 for i in range(len(x1))]
print("Epoch: 1","Weight :",w1,w2,"Bias:",b)
epoch=2
while(y!=actual):
    for i in range(len(x1)):
        z=w1*x1[i]+w2*x2[i]
        y1=step(z)
        error=y[i]-y1
        if(error!=0):
            w1=w1+n*error*x1[i]
            w2=w2+n*error*x2[i]
            print("Sample: ",i+1,"Error: ",w1,w2)
        else:
            print("Sample: ",i+1,"No Error")
        actual[i]=y1
    print("Epoch: ",epoch,"Weight: ",w1,w2,)
    epoch+=1
print("Final Weight: ",w1,w2)
print("Final verification....\n")
for i in range(len(x1)):
    z=w1*x1[i]+w2*x2[i]
    y1=step(z)
    print("Sample: ",i+1,"Output: ",y1)
```

Output:

AND Logic gate

Epoch: 1 Weight : 1.2 0.6 Bias: -0.5

Sample: 1 No Error

Sample: 2 No Error

Sample: 3 Error: 0.7 0.6

Sample: 4 No Error

Epoch: 2 Weight: 0.7 0.6

Sample: 1 No Error

Sample: 2 No Error

Sample: 3 No Error

Sample: 4 No Error

Epoch: 3 Weight: 0.7 0.6

Final Weight: 0.7 0.6

Final verification....

Sample: 1 Output: 0

Sample: 2 Output: 0

Sample: 3 Output: 0

Sample: 4 Output: 1

OR Logic gate

Epoch: 1 Weight : 0.6 0.6

Sample: 1 No Error

Sample: 2 Error: 0.6 1.1

Sample: 3 Error: 1.1 1.1

Sample: 4 No Error

Epoch: 2 Weight: 1.1 1.1

Sample: 1 No Error

Sample: 2 No Error

Sample: 3 No Error

Sample: 4 No Error

Epoch: 3 Weight: 1.1 1.1

Final Weight: 1.1 1.1

Final verification....

Sample: 1 Output: 0

Sample: 2 Output: 1

Sample: 3 Output: 1

Sample: 4 Output: 1

Week-3

A) Write a program to implement classification of linearly separable data with a single layer network

```
import matplotlib.pyplot as plt
x1=[2,3,1,4,5,2]
x2=[3,4,2,1,2,1]
y=[0,0,0,1,1,1]
w1=-0.8
w2=-1.2
n=0.5
t=1
def step(z):
    return 1 if z>=t else 0
actual=[0 for i in range(len(x1))]
print("Epoch: 1","Weight :",w1,w2)
epoch=2
while(y!=actual):
    for i in range(len(x1)):
        z=w1*x1[i]+w2*x2[i]
        y1=step(z)
        error=y[i]-y1
        if(error!=0):
            w1=w1+n*error*x1[i]
            w2=w2+n*error*x2[i]
            print("Sample: ",i+1,"Error: ",w1,w2)
        else:
            print("Sample: ",i+1,"No Error")
        actual[i]=y1
    print("Epoch: ",epoch,"Weight: ",w1,w2)
    epoch+=1
print("Final Weight: ",w1,w2)
plt.scatter(x1, x2, c=y)
plt.show()
```

Output:

Epoch: 1 Weight : -0.8 -1.2

Sample: 1 No Error

Sample: 2 No Error

Sample: 3 No Error

Sample: 4 Error: 1.2 -0.7

Sample: 5 No Error

Sample: 6 No Error

Epoch: 2 Weight: 1.2 -0.7

Sample: 1 No Error

Sample: 2 No Error

Sample: 3 No Error

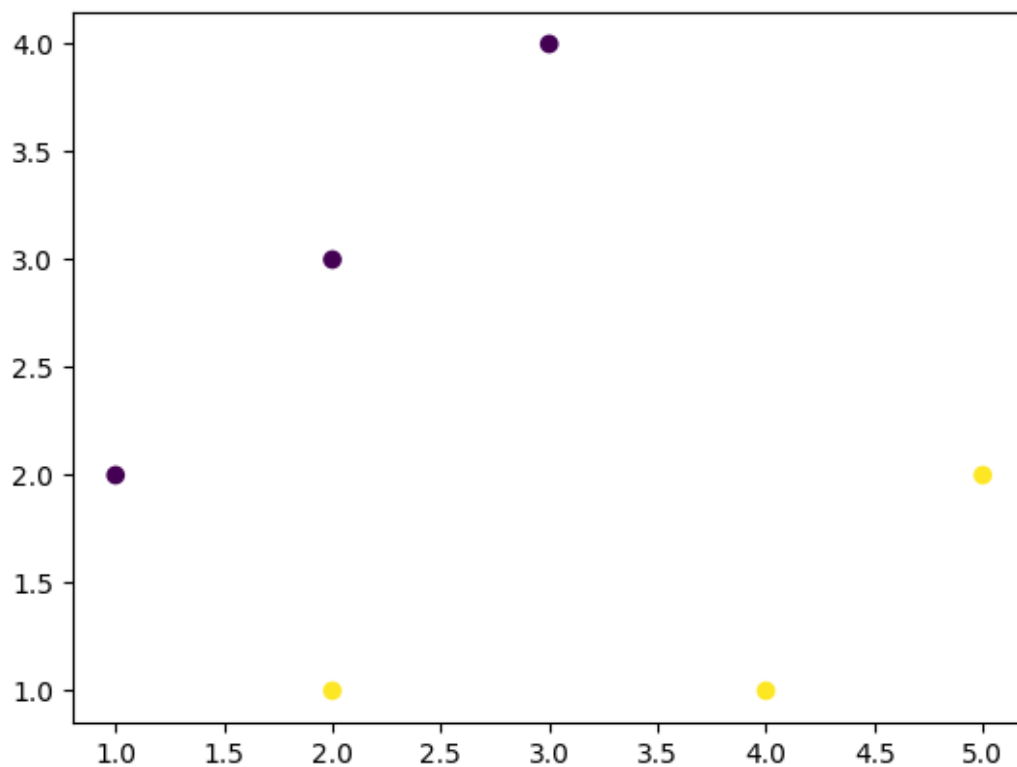
Sample: 4 No Error

Sample: 5 No Error

Sample: 6 No Error

Epoch: 3 Weight: 1.2 -0.7

Final Weight: 1.2 -0.7



Week 4

Write a program to solve XOR problem using back propagation algorithm in multilayer neural networks

```
i. import math
x = [[0,0,1,1], [0,1,0,1]]
y = [0,1,1,0]
w = [[0.2, 0.3], [0.1, 0.34]]
wh = [0.1, 0.2]

n = 0.5

def aj(w,x):
    ans = 0
    for i in range(len(w)):
        ans += w[i]*x[i]

    return ans
def yj(a):
    return 1/(1+math.exp(-a))

print("Epoch: 1","Weights:",w,wh)
epoch = 2

for _ in range(100):
    for i in range(len(y)):
        #forward pass
        h = []
        for j in range(len(w)):
            a = aj(w[j], [x[0][i],x[1][i]])
            h.append(yj(a))

        a = aj(h,wh)
        y1 = yj(a)
        error = y[i] - y1
        if(error !=0):

            #backward pass
            deltay = y1*(1-y1)*(y[i]-y1)

            delta = []
            for j in range(len(w)):
                x1 = h[j] * (1-h[j]) * wh[j] * deltay
                delta.append(x1)

            #update weights
            for j in range(len(wh)):
                x1 = n * deltay * h[j]
                wh[j] = wh[j] + x1

            for q in range(len(w)):
                for j in range(len(w[q])):
                    x1 = n * delta[j] * x[j][i]
                    w[q][j] = w[q][j] + x1
```



```
        print("    Sample:",i+1,"ERROR ", w,wh)
    else:
        print("    Sample:",i+1,"No error")
    print("Epoch:",epoch ,"Weights:",w,wh)
    epoch += 1

print("Final\tWeights:",w,wh)
```

ii. !pip install tensorflow

```
import numpy as np
import tensorflow as tf

input_data = np.array([[0,0],[0,1],[1,0],[1,1]], dtype = np.float32)
target_data = np.array([[0],[1],[1],[0]], dtype = np.float32)

model = tf.keras.Sequential([
    tf.keras.layers.Input(shape = (2,)),
    tf.keras.layers.Dense(2,activation='sigmoid'),
    tf.keras.layers.Dense(1,activation='sigmoid')
])

model.compile(
    optimizer = 'adam',
    loss = 'binary_crossentropy',
    metrics = ['accuracy']
)

model.fit(
    input_data,
    target_data,
    epochs = 100000)

predictions = model.predict(input_data)
print(predictions)
```

Output:

```
Epoch 1/100000
1/1 ————— 0s 418ms/step - accuracy: 0.5000 - loss: 0.7446
Epoch 2/100000
1/1 ————— 0s 24ms/step - accuracy: 0.5000 - loss: 0.7442
Epoch 3/100000
1/1 ————— 0s 24ms/step - accuracy: 0.5000 - loss: 0.7438
Epoch 4/100000
1/1 ————— 0s 21ms/step - accuracy: 0.5000 - loss: 0.7433
Epoch 5/100000
1/1 ————— 0s 23ms/step - accuracy: 0.5000 - loss: 0.7429
Epoch 6/100000
1/1 ————— 0s 22ms/step - accuracy: 0.5000 - loss: 0.7425
Epoch 7/100000
1/1 ————— 0s 23ms/step - accuracy: 0.5000 - loss: 0.7421
Epoch 8/100000
1/1 ————— 0s 20ms/step - accuracy: 0.5000 - loss: 0.7417
Epoch 9/100000
```

Week 5

Write a program to implement Backpropagation in deep Learning.

```
from google.colab import drive
drive.mount('/content/drive')
!ls /content/drive/MyDrive/data/data

import os
import numpy as np
from keras.models import Sequential, Model
from keras.layers import Activation, Dense
from keras import optimizers, applications
import matplotlib.pyplot as plt
from tensorflow.keras.utils import to_categorical

def flatten_images(folder):
    images=[]
    for filename in os.listdir(folder):
        img=plt.imread(os.path.join(folder,filename))
        img = np.array(img)
        img = img.flatten()
        images.append(img)
    return images

airplane = flatten_images('/content/drive/MyDrive/data/airplane')
airplane_data = np.array(airplane)
airplane_data.shape

birds = flatten_images('/content/drive/MyDrive/data/bird')
birds_data = np.array(birds)
birds_data.shape

airplane_labels = np.ones(len(airplane_data))
birds_labels = np.zeros(len(birds_data))
labels = np.concatenate((airplane_labels,birds_labels),axis=0)
labels.shape

features = np.concatenate((airplane_data,birds_data),axis=0)
features.shape
print(features)
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test =
train_test_split(features,labels,test_size=0.3)

model = Sequential([Dense(1024,activation="relu",input_shape=(3072,)),
                    Dense(128,activation="relu"),
                    Dense(32,activation="relu"),
                    Dense(2,activation="softmax")])

import tensorflow as tf
model.compile(
    optimizer="adam",
    loss = "categorical_crossentropy",
    metrics=["accuracy",tf.keras.metrics.Recall]
)
history = model.fit(
```

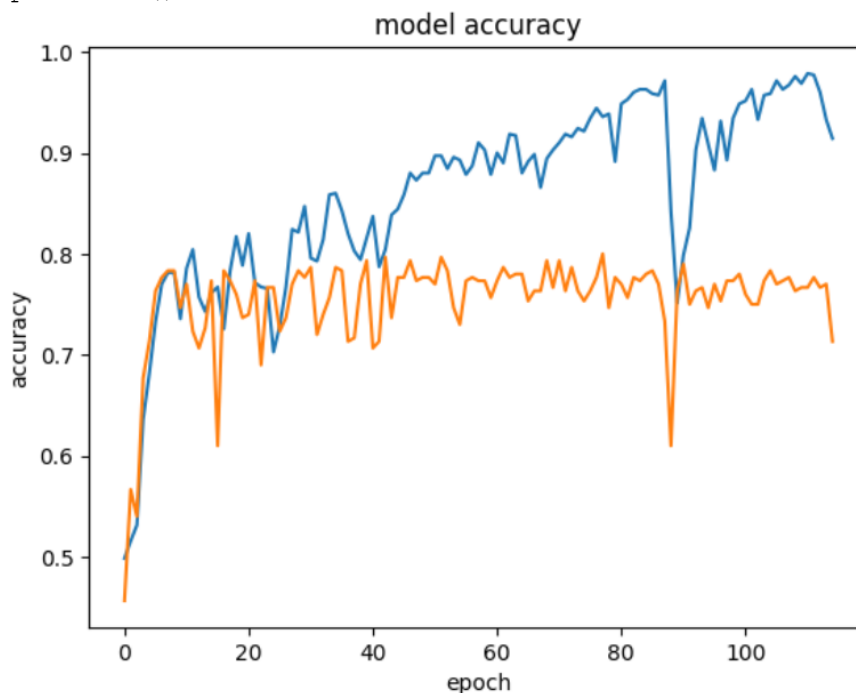
```
X_train,
to_categorical(y_train),
epochs=115,
batch_size=128,
validation_data=(X_test,to_categorical(y_test))
)
Epoch 1/115
6/6 ————— 3s 152ms/step - accuracy: 0.4802 - loss: 1.2191 - recall_1: 0.4802
Epoch 2/115
6/6 ————— 1s 149ms/step - accuracy: 0.5261 - loss: 0.7568 - recall_1: 0.5261
Epoch 3/115
6/6 ————— 1s 136ms/step - accuracy: 0.4890 - loss: 0.7841 - recall_1: 0.4890
Epoch 4/115
6/6 ————— 1s 162ms/step - accuracy: 0.6116 - loss: 0.6512 - recall_1: 0.6116
Epoch 5/115
6/6 ————— 1s 248ms/step - accuracy: 0.6780 - loss: 0.6150 - recall_1: 0.6780
Epoch 6/115
6/6 ————— 2s 96ms/step - accuracy: 0.7195 - loss: 0.5661 - recall_1: 0.7195
Epoch 7/115
6/6 ————— 1s 84ms/step - accuracy: 0.7797 - loss: 0.5140 - recall_1: 0.7797
Epoch 8/115
```

```
results = model.evaluate(X_test,to_categorical(y_test),batch_size=128)
print
print("Test Loss:", results[0])
print("Test Accuracy:", results[1])
print("Test Precision:", results[2])
```

```
3/3 ————— 0s 46ms/step - accuracy: 0.7082 - loss: 0.9930 - recall_1: 0.7082
Test Loss: 0.9591413140296936
Test Accuracy: 0.7133333086967468
Test Precision: 0.7133333086967468
```

```
model.metrics_names
print(history.history.keys())
```

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.show()
```



```
test_folder = "/content/drive/MyDrive/data/test_images"
results=[]
for filename in os.listdir(test_folder):
    temp=[]
    img = plt.imread(os.path.join(test_folder,filename))
    img = np.array(img)
    img = img.flatten()
    img = img.reshape(1,3072)
    prediction = model.predict(img)
    x = np.argmax(prediction)
    if x==0:
        ans = "bird"
    else:
        ans = "airplane"
    temp.append(filename)
    temp.append(ans)
    results.append(temp)
print(results)
```

```
1/1 ————— 0s 77ms/step
1/1 ————— 0s 40ms/step
[['0285.png', 'bird'], ['0093.png', 'bird']]
```

Week 6

Write a program to implement L1 and L2 Regularizations for deep learning models.

```
!pip install tensorflow

import tensorflow as tf
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.regularizers import l1, l2
import matplotlib.pyplot as plt

(x_train, y_train) , (x_test, y_test) = mnist.load_data()
x_train = x_train/255.0
x_test = x_test/255.0

model=Sequential([Flatten(input_shape = (28,28)),
                  Dense(256, activation='relu',
                      kernel_regularizer=l1(0.01),
                      activity_regularizer=l2(0.01)
                  ),
                  Dense(128, activation='relu',
                      kernel_regularizer=l1(0.01),
                      activity_regularizer=l2(0.01)
                  ),
                  Dense(64, activation='relu',
                      kernel_regularizer=l1(0.01),
                      activity_regularizer=l2(0.01)
                  ),
                  Dense(10, activation='softmax')
                ])

model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
history=model.fit(
    x_train,
    y_train,
    epochs=20,
    validation_data=(x_test, y_test)
)

test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print("\nTest Accuracy: ", test_acc)

train_loss, train_acc= model.evaluate(x_train, y_train, verbose=2)
print("\nTrain Accuracy: ", train_acc)

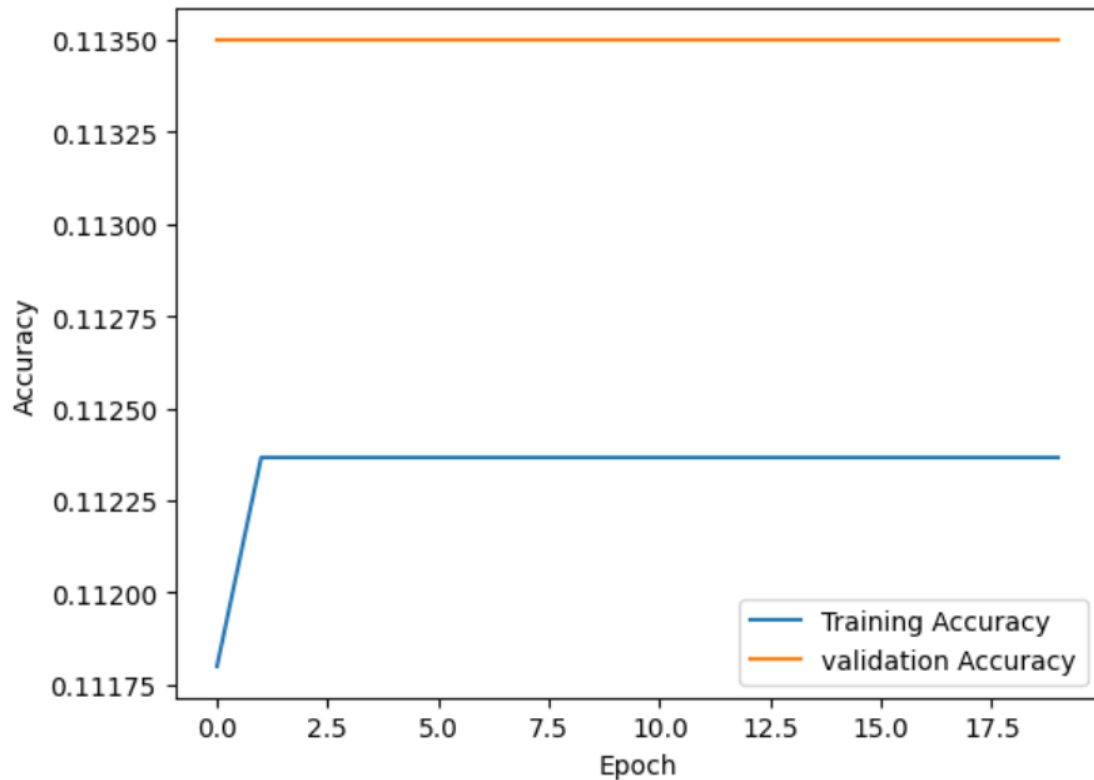
plt.plot(history.history['accuracy'], label = "Training Accuracy")
plt.plot(history.history['val_accuracy'], label = "validation Accuracy")
plt.xlabel('Epoch')
```

```
plt.ylabel('Accuracy')  
plt.legend()  
plt.show()
```

Output:

Test Accuracy: 0.11349999904632568
1875/1875 - 6s - 3ms/step - accuracy: 0.1124 - loss: 2.5958

Train Accuracy: 0.11236666887998581



Week 7

Implement the following optimization algorithms on deep neural networks

a) Stochastic Gradient Descent b) Ada Grad c) Adam

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.optimizers import SGD, Adagrad, Adam
from tensorflow.keras.utils import to_categorical

(x_train,y_train),(x_test,y_test)=mnist.load_data()

x_train=x_train.reshape(-1,28*28)/255.0
x_test=x_test.reshape(-1,28*28)/255.0

y_train=to_categorical(y_train,10)
y_test=to_categorical(y_test,10)

def create_model():
    model=tf.keras.Sequential([
        tf.keras.layers.Dense(128,activation='relu',input_shape=(784,)),
        tf.keras.layers.Dense(64,activation='relu'),
        tf.keras.layers.Dense(10,activation='softmax')
    ])
    return model

epochs=10
batch_size=64

# SGD
model=create_model()
sgd_optimizer=SGD(learning_rate=0.01)
model.compile(optimizer=sgd_optimizer,loss='categorical_crossentropy',metrics=['accuracy'])
sgd_history=model.fit(x_train,y_train,epochs=epochs,batch_size=batch_size,validation_data=(x_test,y_test))

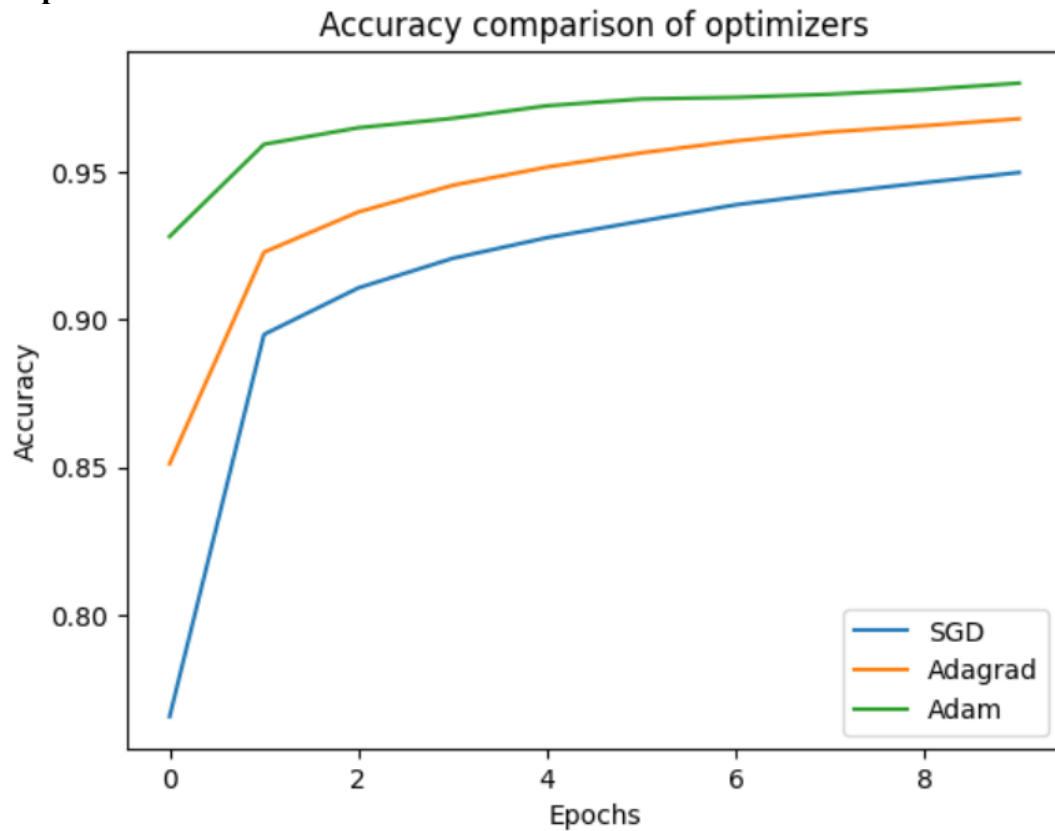
# Adagrad
model=create_model()
adagrad_optimizer=Adagrad(learning_rate=0.01)
model.compile(optimizer=adagrad_optimizer,loss='categorical_crossentropy',metrics=['accuracy'])
adagrad_history=model.fit(x_train,y_train,epochs=epochs,batch_size=batch_size,validation_data=(x_test,y_test))

# Adam
model=create_model()
adam_optimizer=Adam(learning_rate=0.01)
model.compile(optimizer=adam_optimizer,loss='categorical_crossentropy',metrics=['accuracy'])
adam_history=model.fit(x_train,y_train,epochs=epochs,batch_size=batch_size,validation_data=(x_test,y_test))
```

```
# Plot
plt.plot(sgd_history.history['accuracy'],label='SGD')
plt.plot(adagrad_history.history['accuracy'],label='Adagrad')
plt.plot(adam_history.history['accuracy'],label='Adam')

plt.title('Accuracy comparison of optimizers')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

Output:



Week 8

Write a program to implement Convolution neural network on any image dataset.

```
!pip install tensorflow

import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import numpy as np

(train_img, train_lab), (test_img, test_lab) =
datasets.mnist.load_data()
train_img.shape

train_img = train_img.reshape((60000, 28, 28, 1)) / 255.0
test_img = test_img.reshape((10000, 28, 28, 1)) / 255.0

model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28,
1)),
    layers.MaxPooling2D((2,2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='relu')
])

model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
model.fit(train_img, train_lab , epochs = 5, validation_data=(test_img,
test_lab))
test_loss, test_acc = model.evaluate(test_img, test_lab)
print("Test loss:",test_loss,"\nTest Accuracy:",test_acc)
```

Output:

Test Loss: 2.30259108543396

Test Accuracy: 0.09799999743700027

Week 9

Write a program to find the similarity between word documents using neural networks

```
import tensorflow as tf
import tensorflow_hub as hub
module_url="https://tfhub.dev/google/universal-sentence-encoder/4"
embed=hub.load(module_url)
def calculate_similarity(doc1,doc2):
    embeddings=embed([doc1,doc2])
    print("Embedding:",embeddings)
    similarity=tf.keras.losses.cosine_similarity(embeddings[0],embeddings[
1])
    return similarity
def print_similarity(doc1,doc2):
    similarity=calculate_similarity(doc1,doc2)
    similarity_score=calculate_similarity(doc1,doc2)
    print(f"\nSimilarity Score:{similarity_score:.4f}")

doc1="A fast brown fox jumps over the lazy dog"
doc2="A quick brown fox jumps over the lazy dog"
print_similarity(doc1,doc2)
```

Output:

Similarity Score:-0.9609

Week 10

Write a program to build speech to text model using deep neural networks

```
!pip install SpeechRecognition
!pip install pyttsx3

import speech_recognition as sr
import pyttsx3
import requests
from requests.structures import CaseInsensitiveDict

recognizer = sr.Recognizer()
def addPunch(mytext):
    url = "http://bark.phon.ico.ee/punctuator"
    headers = CaseInsensitiveDict()
    headers["Content-Type"] = "application/x-www-form-urlencoded"
    data = f"text = {mytext}"
    resp = requests.post(url, headers = headers, data = data)
    mytext = resp.text
    return mytext

audio_file = 'harvard.wav'
with sr.AudioFile(audio_file) as source:
    recognizer.adjust_for_ambient_noise(source)
    try:
        audio_data = recognizer.record(source)
        text = recognizer.recognize_google(audio_data)
        text = addPunch(text)
        print('Transcription:',text)
    except sr.UnknownValueError:
        print('Speech Recognition could not understand audio')
    except sr.RequestError as e:
        print(f'Could not request results from Speech Recognition service; {e}')
    except Exception as e:
        print(e)
```

Output:

Transcription: Does snail smell of old beer lingers? It takes heat to bring out the odor; a cold dip restores health and zest. A salt pickle taste fine with ham. Tacos al pastor are my favorite. A zestful food is the hot cross bun.